How does web architecture shape scalability?

Lecture 4 · Challenge

Architecture determines where work, state, authority, and failure can move.

Architecture is a set of consequential boundaries

An architecture decides:

- where computation runs;
- which representation crosses each interface;
- where authoritative state lives;
- who may perform an operation;
- how failures and changes propagate;
- which team owns each contract.

A diagram is useful only when it exposes those decisions.

Four roles clarify a web request

Role	Architectural question
Consumer	Who or what initiates the interaction?
Representation	Which document, message, or interface crosses the boundary?
Computation	Where is behavior evaluated?
Data	Where is state authoritative, copied, indexed, or derived?

One component may perform several roles. Keep the responsibilities conceptually separate.

Trace the request before naming the pattern

For one important operation, follow:

consumer → network boundary → representation → application logic → authority check → data read/write → response

At every hop, ask:

- What contract is assumed?
- What can fail or become slow?
- Which state is local, copied, or authoritative?
- Who owns compatibility and recovery?

Rendering moves work across time and place

SSR, SSG, ISR, and CSR primarily describe **where and when a view is produced or refreshed**.

SPA, MPA, and hypermedia patterns primarily describe **how the browser reaches later application states**.

Compare their consequences for freshness, cacheability, client work, accessibility, navigation, origin load, and failure.

Automated consumers change both demand and exposure

Distinguish:

- cooperative crawlers and monitors;
- authorized integrations and agents;
- unknown automation;
- abusive bots and scraping.

`robots.txt` coordinates cooperative crawling; it is not access control.

Authentication, authorization, rate policy, data minimization, and useful error responses protect application boundaries.

Organizational boundaries become software contracts

Interfaces often mirror communication and ownership structures.

Ask:

- Which team can change this contract?
- Who coordinates a breaking change?
- Where does cross-team review become a bottleneck?
- Which service boundary exists for organizational convenience rather than runtime pressure?

Team topology and deployment topology are related, but they are not the same design.

Launch Challenge with an architecture hypothesis

For implementation B, record:

- one request and data path;
- authority and ownership boundaries;
- the automated-consumer policy;
- one architectural difference from A;
- the pressure that difference should improve;
- the cost or failure mode it may introduce.

Question to carry forward: How difficult is this architecture for a newcomer to change safely?